Voyage-AI — Implementation Status & Feature Inventory

Document Reference: `docs/reports/implementation-status.pdf` / `docs/reports/implementation-status.md`
System Version: 0.1.0 (Phase 1 Baseline)
Author: Senior Systems Analyst & QA Engineer
Audit Date: October 21, 2026
Status: Implementation-Verified

---

1. Status Classification Definitions

Every significant component, feature, and subsystem in Voyage-AI is classified according to the following strict evidence-based criteria:

- **Implemented and verified:** The implementation exists in active source code and has been conclusively verified through passing automated test execution or runtime inspection evidence in this session.
- **Implemented but unverified:** The implementation exists in active source code, but automated tests or runtime verification have not been executed or are blocked by environment prerequisites.
- **Partially implemented:** Code exists for a portion of the feature (e.g., frontend prototype or backend adapter), but the complete end-to-end operational workflow is incomplete.
- **Planned:** Formally specified in architecture or requirement documentation (e.g., `PRD.md`, `TDD.md`, `JWT_AUTH_DESIGN.md`), but no active implementation code exists in the repository.
- **Deprecated or superseded:** Previously proposed, documented, or implemented, but explicitly replaced by newer architecture or discarded.

---

2. Comprehensive Implementation Status Inventory

Component or Feature	Status	Evidence (Source Files)	Verification Performed
Vite & React Application Shell	Implemented and verified	`frontend/src/App.tsx`, `frontend/src/main.tsx`, `frontend/package.json`	Executed `npm run test:unit` (7/7 passed); ins Vite
Tactile Button Component	Implemented and verified	`frontend/src/components/common/Button.tsx`	Tested in `App.test.tsx`; visual inspection of `index.css`
Semantic Tag Badges	Implemented and verified	`frontend/src/components/common/Badge.tsx`	Tested in `App.test.tsx`; verified rendering of r
Accessible Shimmer Skeletons	Implemented and verified	`frontend/src/components/common/SkeletonCard.tsx`	Rendered during filter debounce (120ms); tested i
Zero-Results Empty State	Implemented and verified	`frontend/src/components/common/EmptyState.tsx`	Tested in `DestinationsDiscoveryTest.java` action
Image Loader with Fallback	Implemented and verified	`frontend/src/components/common/ImageWithFallback.tsx`	Verified via `ImageWithFallback.test.tsx` (3/ image error)
Editorial Navbar & Brand Header	Implemented and verified	`frontend/src/components/layout/Navbar.tsx`	Tested in `App.test.tsx`; verified live health pol
Editorial Hero Showcase	Implemented and verified	`frontend/src/components/discovery/HeroSection.tsx`	Tested in `App.test.tsx`; verified WCAG 2.2 AA click
Multi-Attribute Filter Bar	Implemented and verified	`frontend/src/components/discovery/FilterBar.tsx`	Tested in `App.test.tsx` and `DestinationsDi` style, budget filters)
Destination Card Component	Implemented and verified	`frontend/src/components/discovery/DestinationCard.tsx`	Tested in `App.test.tsx`; verified rendering of c daily budget
5-Tab Detail Inspection Modal	Implemented and verified	`frontend/src/components/discovery/DestinationDetailModal.tsx`	Tested in `App.test.tsx` (ESC key dismissal, clo
Curated Benchmark Dataset	Implemented and verified	`frontend/src/data/mockDestinations.ts`	Inspected on disk; verified across Jaipur, Varanasi
Backend Application Bootstrap	Implemented and verified	`backend/src/main/java/com/voyageai/VoyageAiApplication.java`	Spring Boot context successfully boots in `Voyage`

Component or Feature	Status	Evidence (Source Files)	Verification Performed
Uniform API Response Envelope	Implemented and verified	`backend/src/main/java/com/voyageai/common/api/ApiResponse.java`	Verified in `HealthControllerTest.java` and
Global Exception Handler	Implemented and verified	`backend/src/main/java/com/voyageai/common/api/GlobalExceptionHandler.java`	Verified in `WeatherControllerTest.java` for
Service Health Check API	Implemented and verified	`backend/src/main/java/com/voyageai/common/health/HealthController.java`	Verified via `HealthControllerTest.java` (20 `HealthApiTest.java`
Spring Boot Actuator Health	Implemented and verified	`backend/pom.xml`, `backend/src/main/resources/application.yml:33`	Verified via `HealthControllerTest.actuatorHealthEndp`
Cross-Origin Security (CORS)	Implemented and verified	`backend/src/main/java/com/voyageai/config/CorsConfig.java`	Inspected configuration; verified cross-origin calls
Open-Meteo Weather Service	Implemented and verified	`backend/src/main/java/com/voyageai/integration/weather/OpenMeteoWeatherService.java`	Verified via `OpenMeteoWeatherServiceTest.j`
Weather REST Controller	Implemented and verified	`backend/src/main/java/com/voyageai/integration/weather/WeatherController.java`	Verified via `WeatherControllerTest.java` (7/ `WeatherApiTest.java`
Flyway Baseline Migration	Implemented and verified	`backend/src/main/resources/db/migration/V0__baseline.sql`	Verified via `DatabaseFlywayIntegrationTest` memory database
Database Isolation Guard	Implemented and verified	`backend/src/test/java/com/voyageai/integration/db/DatabaseFlywayIntegrationTest.java`	Executed during `mvn test` (asserts developer d
Docker Compose Orchestration	Implemented and verified	`infra/docker-compose.yml`, `infra/init-db.sql`	Inspected port mapping `3307:3306`; verified he
Centralized Playwright Suites	Implemented but unverified (in this session)	`tests/src/test/java/com/voyageai/tests/browser/`, `tests/src/test/java/com/voyageai/tests/api/`	Previously verified in `run-all-tests.ps1` logs servers were not booted
Domain Relational Persistence	Planned (Phase 2 Target)	Specified in `Architecture.md` and `IMPLEMENTATION_PLAN.md`	Flyway V0 baseline ready; schema designed for
Stateless JWT Authentication	Planned (Phase 2 Target)	Specified in `docs/JWT_AUTH_DESIGN.md`; placeholder in `application.yml:48`	Architecture approved; BCrypt and Spring Security
Interactive Leaflet Map Tiles	Planned (Phase 2 Target)	Specified in `docs/DATA_AUTHENTICITY_AND_PROVIDER_PLAN.md`	Policy verified against OSM usage guidelines; UI ir
Wikimedia Media Ingestion	Planned (Phase 2 Target)	Specified in `docs/DATA_AUTHENTICITY_AND_PROVIDER_PLAN.md`	User-Agent policy verified; image fallback compor
OSRM Transit Distance Routing	Planned (Phase 3 Target)	Specified in `docs/DATA_AUTHENTICITY_AND_PROVIDER_PLAN.md`	Provider interface abstraction designed; integratio
Grounded Gemini 1.5 Flash AI	Planned (Phase 5 Target)	Config placeholder in `application.yml:53`; specified in `DATA_AUTHENTICITY_PLAN.md`	AI Studio free tier limits verified (15 RPM); zero PII
Deprecated AI Provider Variables	Deprecated or superseded	Commit `97030b6`	Commit record confirms removal of obsolete AI pr

3. Discrepancy & Inconsistency Analysis

3.1 Existing Documentation vs. Implementation Realities

1. **JPA Entity Existence:**
2. *Documentation Discrepancy:* Earlier specifications (e.g., initial drafts of `PRD.md` and `TDD.md`) occasionally implied that `User`, `Destination`, and `Trip` database entities were operational in Phase 0.
3. *Actual Implementation State:* Spring Data JPA repository scanning logs during `task-121.log` explicitly confirm: `Finished Spring Data repository scanning in 61 ms. Found 0 JPA repository interfaces.` The database currently contains only the Flyway `V0__baseline.sql` migration.

4. *Resolution:* Documentation accurately reflects that domain entities are scheduled for Phase 2.
5. **Frontend Data Source:**
6. *Documentation Discrepancy:* High-level marketing descriptions suggest that the destination catalog is dynamically fetched from backend MySQL database queries.
7. *Actual Implementation State:* Inspection of `App.tsx:9` confirms that discovery catalog rendering is powered directly by `MOCK_DESTINATIONS` from `mockDestinations.ts`. Backend HTTP traffic from the client is currently restricted to `/api/v1/health` connectivity polling.
8. *Resolution:* Correctly documented as Phase 1 Discovery Prototype architecture.
9. **External Weather API Dependency:**
10. *Documentation Discrepancy:* `README.md` documents live Open-Meteo weather forecasts at `http://localhost:8081/api/v1/weather?latitude=...&longitude=...`.
11. *Actual Implementation State:* Fully verified. Both MockMvc unit tests and direct Spring Boot controller implementations confirm that `/api/v1/weather` is fully implemented and operational.

---

4. Closing Outstanding Verification Gaps

To advance unverified and planned components to **Implemented and verified**, the following evidence must be generated in upcoming phases:
1. **Executing the Centralized Playwright Suite:** Boot the backend (`mvn spring-boot:run` on port 8081) and frontend (`npm run dev` on port 5173), then execute `mvn test` in `tests/` to collect fresh Playwright execution logs.
2. **Phase 2 Relational Persistence Delivery:** Author `V1__init_schema.sql`, author Spring Data JPA entity classes (`User`, `Destination`, `Trip`), and write JPA repository integration tests executing against the Flyway schema.
3. **Phase 2 JWT Security Implementation:** Author `JwtAuthenticationFilter`, `TokenProvider`, and authentication controllers (`/api/v1/auth/login`, `/api/v1/auth/register`) backed by MockMvc test suites asserting 401 unauthorized on protected routes.